Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей
Кафедра: Электронных вычислительных машин
Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО.
КАЛЬКУЛЯТОР ЧИСЛЕННОСТИ НАСЕЛЕНИЯ»
Вариант 7

Студент: Н.И. Волкович

Преподаватель: О.М. Внук

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – освоить практические приёмы работы в среде разработки IntelliJ IDEA при создании объектно-ориентированного приложения на языке Java, научиться проектировать и отлаживать программы с графическим интерфейсом, а также закрепить навыки применения архитектурного паттерна MVC (с активной моделью) на примере реальной задачи.

Задачи:

1. Установить и настроить IntelliJ IDEA, ознакомиться с её основными возможностями: редактором кода, средствами сборки, запуска и отладки программ;
2. Подобрать и настроить инструменты для работы с UML- и SysML v2-диаграммами в выбранной среде разработки, зафиксировать результат в отчёте;
3. Создать пустой Java-проект и разработать в нём приложение «Калькулятор населения мира» в соответствии с вариантом №7;
4. Реализовать графический интерфейс: главное окно с кнопкой вызова окна ввода и отдельное диалоговое окно для ввода даты (без использования стандартного компонента выбора даты);
5. Обеспечить сохранение последних введённых данных и их автоматическое восстановление при повторном открытии программы;
6. Добавить проверку корректности вводимых данных с выводом понятного сообщения об ошибке;
7. Разделить приложение на три компонента (Model, View, Controller) и реализовать активную модель, при которой модель самостоятельно уведомляет представление об изменении данных;
8. Протестировать основные сценарии работы программы и подготовить ответы на контрольные вопросы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Что такое интегрированная среда разработки

Интегрированная среда разработки (IDE) – это программа, которая объединяет в себе всё необходимое для создания кода: редактор, компилятор, отладчик, инструменты сборки и многое другое. Раньше программистам приходилось использовать отдельные утилиты для каждой задачи, но современные IDE позволяют делать всё в одном окне. В IntelliJ IDEA код анализируется прямо во время набора – среда сразу подчёркивает ошибки, предлагает исправления и позволяет запустить программу, не выходя из редактора.

2.2 Работа с кодом в IntelliJ IDEA

Редактор IntelliJ IDEA – это не просто блокнот для текста. Он умеет предугадывать, что ты хочешь написать, автоматически подставляет нужные классы, следит за форматированием и помогает переименовывать переменные без страха что-то сломать. Основные горячие клавиши, которые я использовала в работе: Ctrl+B – перейти к объявлению, Alt+F7 – найти все использования, Ctrl+Alt+L – отформатировать код, Alt+Enter – показать возможные исправления. Структура проекта видна в окне Project, а сообщения компилятора и вывод программы – в окнах Build и Run.

2.3 Как программа собирается, запускается и отлаживается

Java-код не выполняется напрямую – сначала компилятор javac превращает его в байт-код, который потом исполняет виртуальная машина Java (JVM). В IntelliJ IDEA для сборки проекта достаточно нажать Build Project, а для запуска – использовать кнопки Run или Debug. Если нужно найти ошибку, я ставила точку останова (кликом слева от строки или через Ctrl+F8). Когда программа останавливалась, в окне Debug я видела значения всех переменных и могла выполнять код по шагам с помощью команд Step Over, Step Into и Step Out.

2.4 Архитектурный паттерн MVC

MVC (Model-View-Controller) – это способ разделить программу на три части. Model хранит данные и выполняет все расчёты. View отвечает за то, что видит пользователь – окна, кнопки, метки. Controller принимает действия пользователя (нажатия кнопок, ввод текста) и решает, что с ними делать.

Благодаря такому разделению код становится понятнее, а тестировать логику можно отдельно от интерфейса.

2.5 Активная и пассивная модель

В пассивной модели контроллер сам следит за изменениями и обновляет представление. В активной модели всё иначе – модель сама сообщает всем заинтересованным сторонам, что её данные изменились. В моей программе Model использует паттерн «Наблюдатель» (Observer): она хранит список слушателей через интерфейс ModelListener и уведомляет их при каждом пересчёте населения.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

В соответствии с вариантом №7 была разработана программа «Калькулятор населения мира». Программа принимает дату в формате ДД.ММ.ГГГГ и вычисляет приблизительную численность населения Земли на этот день. На главной форме расположена кнопка «Ввести данные», по нажатию на которую открывается отдельное модальное окно InputDialog для ввода даты. Стандартный компонент выбора даты не использовался – ввод осуществляется вручную через текстовое поле. Последние введённые данные сохраняются в файл last_data.txt и автоматически подставляются при повторном открытии программы.

3.2 Настройка среды и расширений для UML и SysML v2

Проект создан в IntelliJ IDEA Community Edition. В качестве JDK выбран Temurin-17.

Для работы с UML был установлен плагин plantuml4idea (версия для IntelliJ IDEA 2025.3.2). Плагин позволяет строить диаграммы классов прямо в среде разработки с использованием текстовой нотации PlantUML. На рисунке 3.1 показано окно установленных плагинов с активным plantuml4idea.

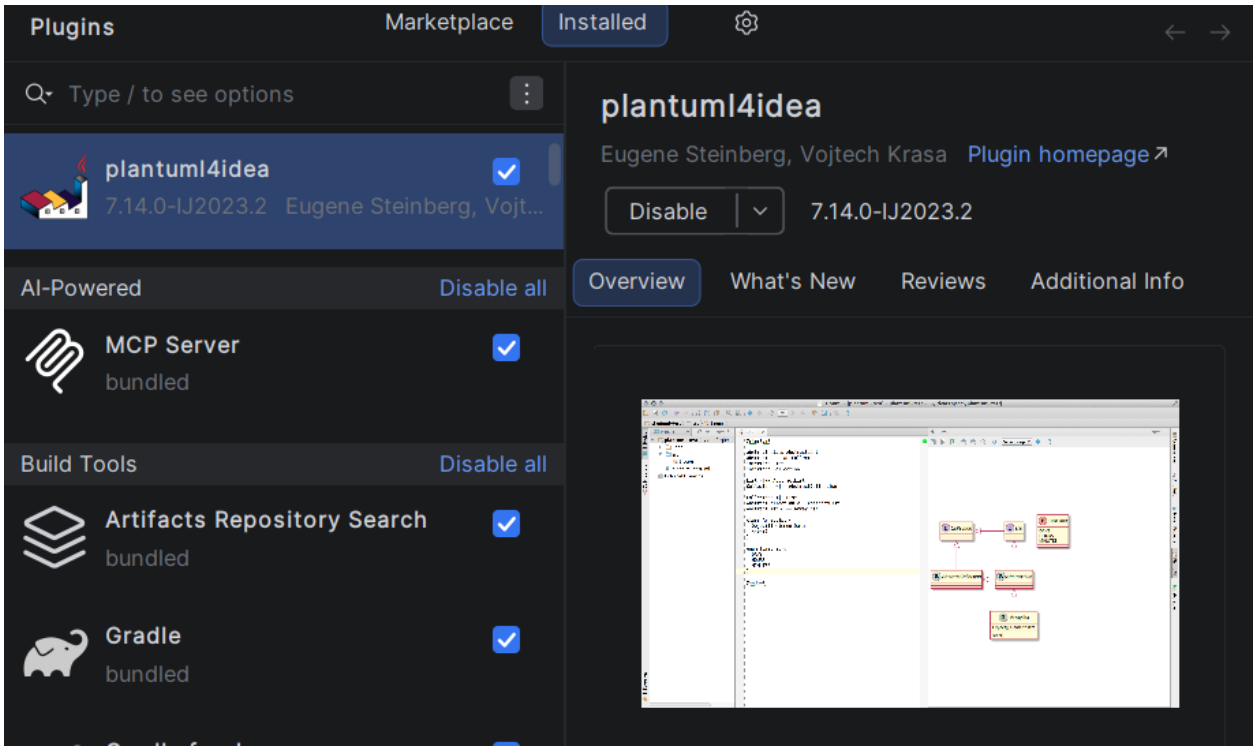


Рисунок 3.1 – Плагин plantuml4idea для работы с UML

Для работы с SysML v2 использовалась текстовая нотация языка – структурное описание системы приведено ниже:

```
package PopulationCalculator {
    part def Model;
    part def View;
    part def InputDialog;
    part def Controller;

    part application {
        part model : Model;
        part view : View;
        part dialog : InputDialog;
        part controller : Controller;
        connect view to controller;
        connect controller to model;
        connect model to view;
    }
}
```

3.3 Демонстрация отладчика

Работа отладчика продемонстрирована в методе calculatePopulation() класса Model. Точка останова установлена на строке 45 – вызове метода interpolateByDay(), который выполняет интерполяцию между двумя ближайшими историческими точками.

С помощью функции Evaluate Expression проверялась корректность промежуточных вычислений. Пошаговое выполнение (Step Over) позволило проследить, как программа выполняет линейную интерполяцию между двумя известными точками.

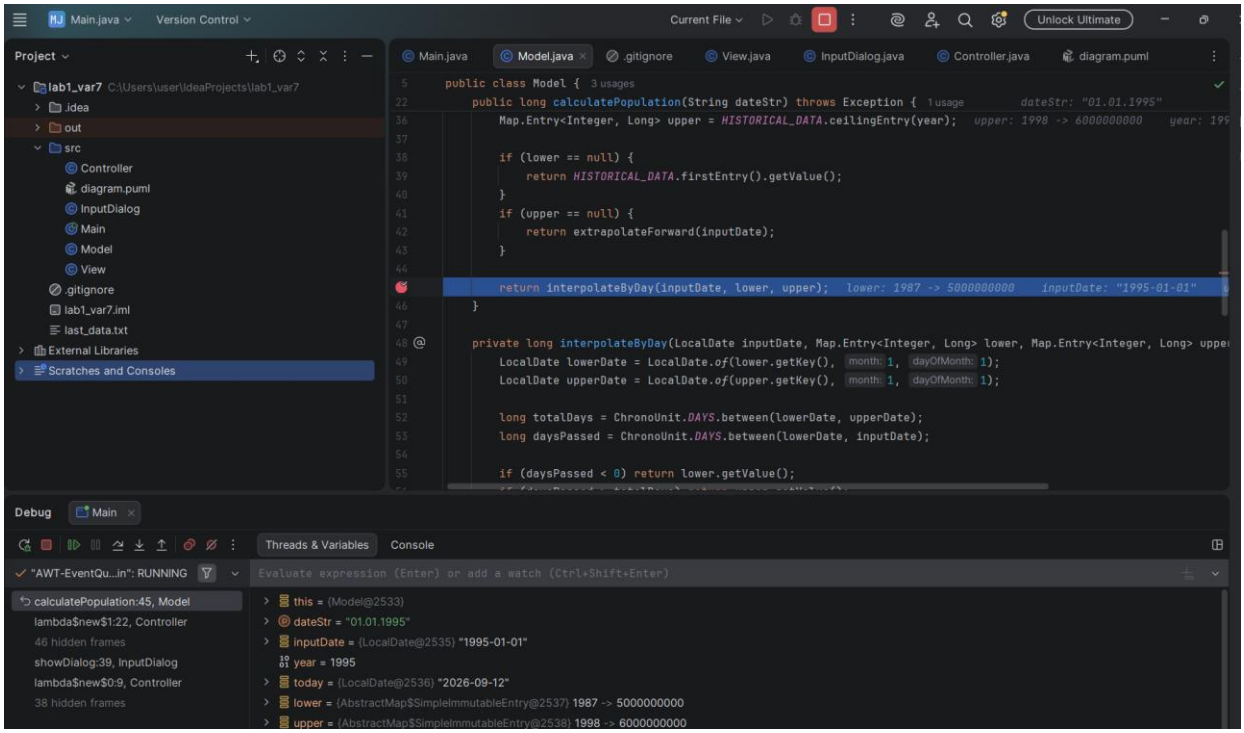


Рисунок 3.2 – Остановка программы в отладчике IntelliJ IDEA

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на следующие компоненты:
– `Model` – хранит исторические данные о населении мира, выполняет интерполяцию и экстраполяцию, проверяет корректность даты и напрямую обновляет представление через активную модель;
– `View` – главное окно с кнопкой «Ввести данные», меткой результата и меткой последнего ввода; отвечает за сохранение данных в файл `last_data.txt` и их восстановление при запуске;
– `InputDialog` – модальное окно для ввода даты в текстовое поле;
– `Controller` – обрабатывает нажатия кнопок, получает дату из диалога, вызывает метод модели и закрывает окно;
– `Main` – точка входа, создающая экземпляры всех классов и связывающая их между собой.

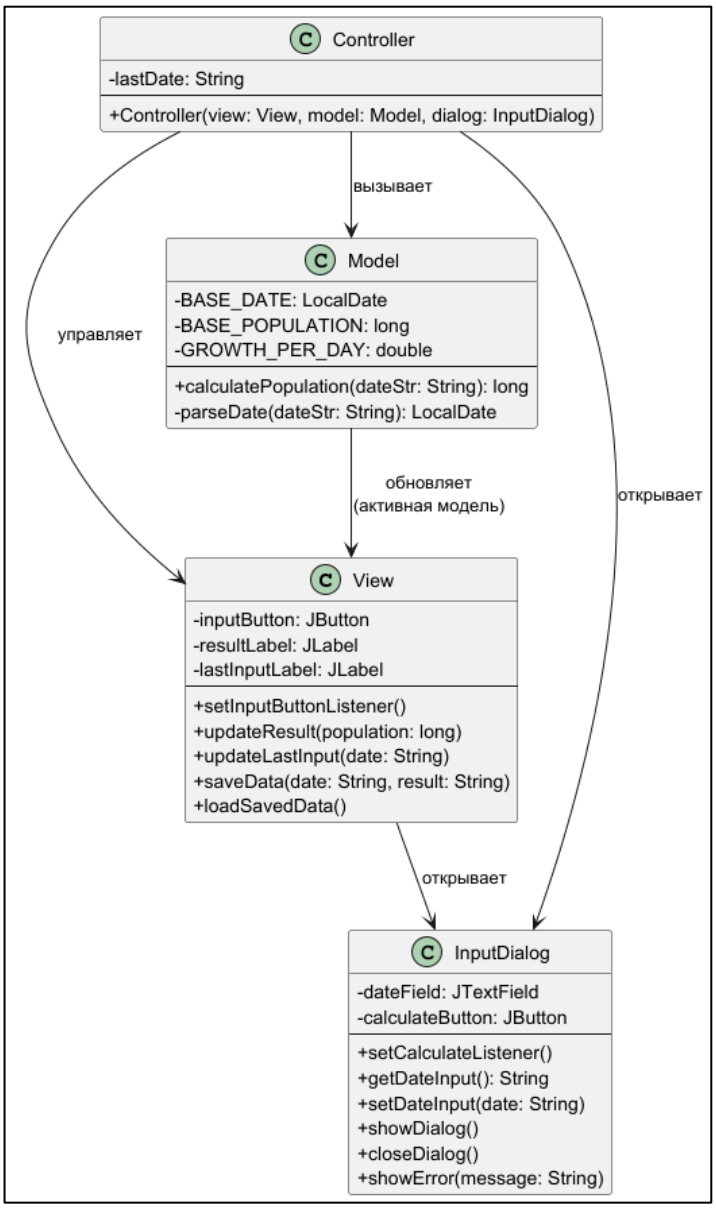


Рисунок 3.3 – Диаграмма классов приложения (MVC с активной моделью)

3.5 Модель

Класс Model реализует активную модель. Внутри хранится TreeMap<Integer, Long> с историческими данными о населении мира (1600, 1700, 1800, 1900, 1950, 1959, 1974, 1987, 1998, 2011, 2025). Метод calculatePopulation() выполняет следующие действия:

1. Вызывает parseDate() для разбора строки и проверки корректности даты (день 1–31, месяц 1–12, существование даты);
2. Сравнивает введённую дату с реальной и запрещает ввод дат в будущем;
3. Если год есть в таблице – возвращает точное значение;
4. Если год между известными точками – вызывает interpolateByDay(), которая выполняет линейную интерполяцию с точностью до дня;
5. Если год после 2025 – вызывает extrapolateForward() (прирост ~80 млн человек в год);
6. После расчёта напрямую вызывает view.updateResult()– это и есть реализация активной модели.

3.6 Представление

Класс View отображает главное окно с кнопкой «Ввести данные» и двумя метками: для результата и для последнего ввода. При нажатии кнопки открывается диалоговое окно InputDialog. После расчёта модель вызывает метод, который обновляет метку результата и сохраняет данные в файл. При запуске программы метод loadSavedData() читает файл и восстанавливает последние данные на главной форме.

Класс InputDialog представляет собой модальное окно с текстовым полем для ввода даты и кнопкой «Рассчитать». При открытии окна в поле автоматически подставляется последняя введённая дата.

3.7 Контроллер

Класс Controller принимает ссылки на View, Model и InputDialog. В конструкторе назначаются обработчики: для кнопки «Ввести данные» – открытие диалога с предварительной подстановкой последней даты; для кнопки «Рассчитать» — получение даты из диалога, вызов метода model.calculatePopulation() и закрытие окна.

3.8 Запоминание данных и обработка ошибок

Приложение сохраняет последнюю введённую дату и результат в файл last_data.txt. При повторном запуске эти данные загружаются методом loadSavedData() и отображаются на главной форме.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В IntelliJ IDEA создано и проверено Java-приложение по варианту №7 – «Калькулятор населения мира». Программа вычисляет численность населения Земли на заданную дату, сохраняет последнее успешное состояние в файл last_data.txt и восстанавливает его при повторном запуске, а также сообщает пользователю о некорректном вводе.

Архитектура приложения соответствует паттерну MVC с активной моделью: контроллер обрабатывает действия пользователя, модель выполняет расчёт численности населения и напрямую уведомляет представление через вызов view.updateResult().

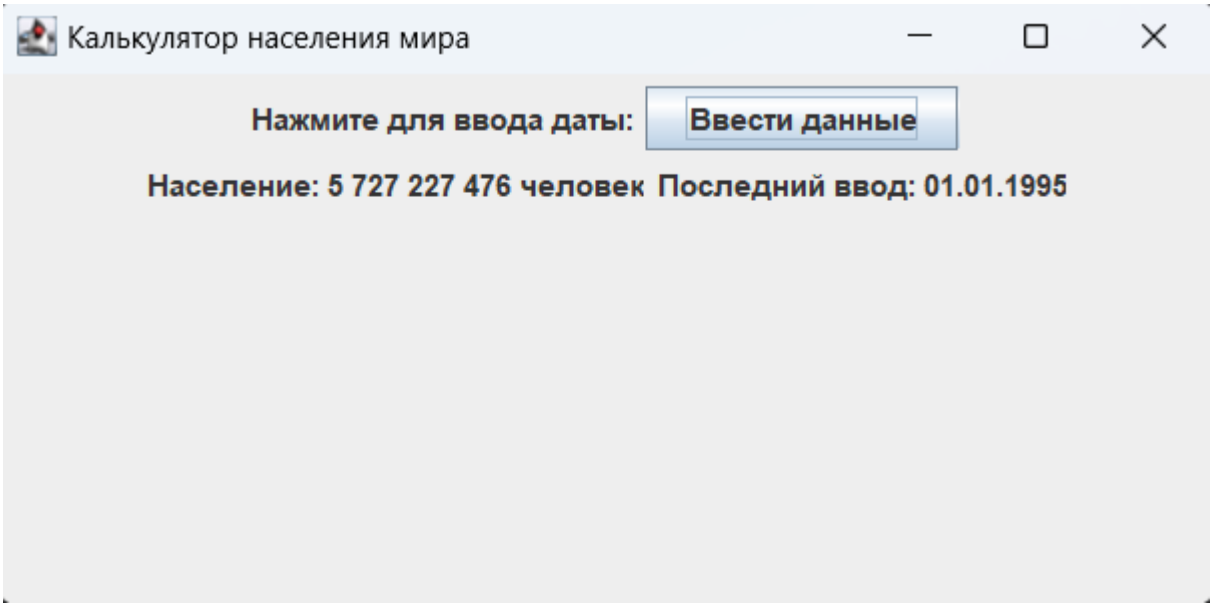


Рисунок 4.1 – Главная форма после запуска

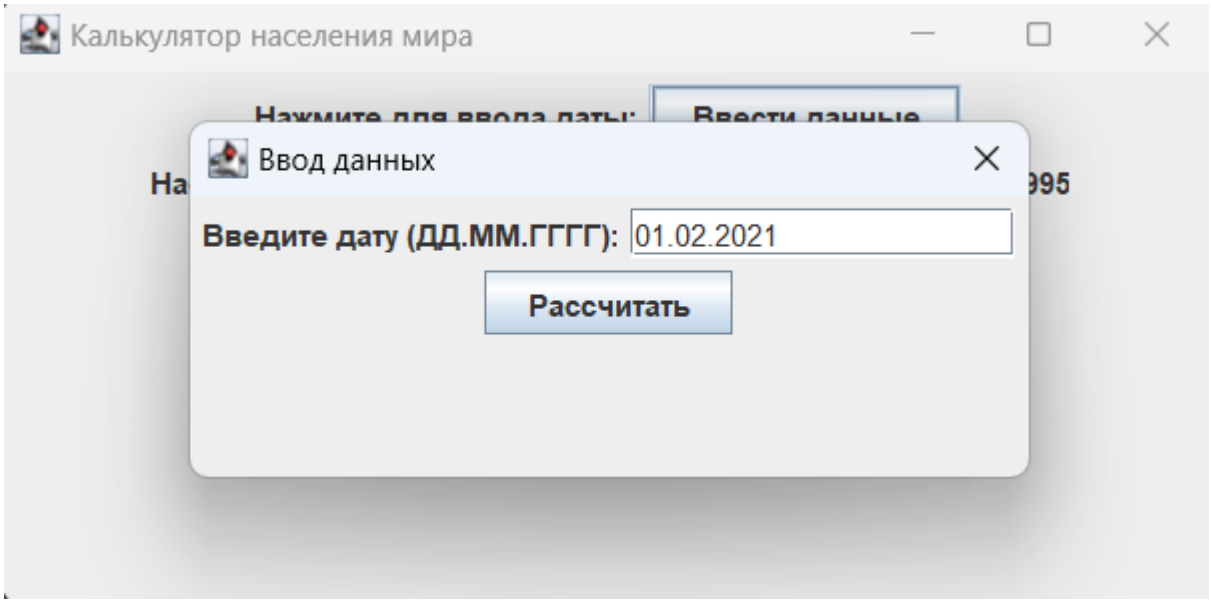


Рисунок 4.2 – Окно ввода данных

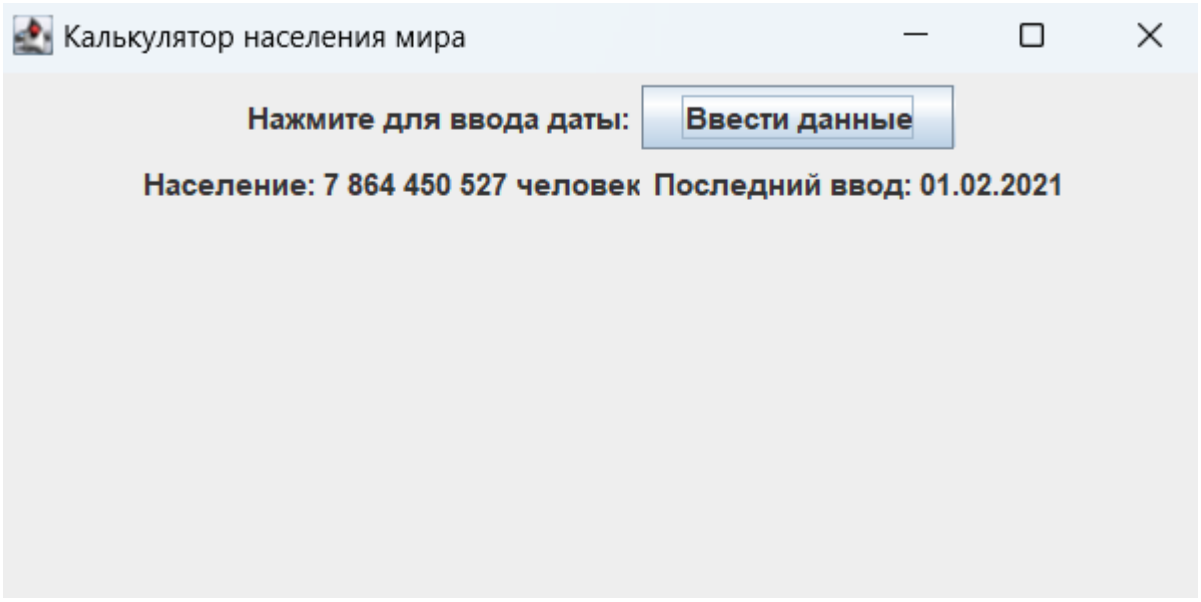


Рисунок 4.3 – Результат вычисления

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

В состав IDE входят: редактор кода, компилятор или интерпретатор, система сборки и запуска, отладчик, средства навигации и рефакторинга, терминал, интеграция с системой контроля версий, а также механизм расширений (плагинов). В IntelliJ IDEA основные рабочие окна называются Project, Build, Run, Debug, Problems и Terminal.

Вопрос 2 Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся: автодополнение (Ctrl + Space), быстрые исправления (Alt + Enter), автоматический импорт классов, форматирование (Ctrl + Alt + L), переход к объявлению (Ctrl + B), поиск использований (Alt + F7), переименование (Shift + F6), просмотр иерархии классов. Эти операции позволяют изменять код быстрее и уменьшают вероятность механических ошибок.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

IntelliJ IDEA компилирует Java-файлы командой Build Project. Для запуска выбирается класс App с методом public static void main(String[] args) и выполняется команда Run. При запуске в режиме Debug программа работает под управлением отладчика и останавливается на заданных точках. Проект также можно собрать командой javac --release 17 и запустить командой java с указанием класса by.bsuir.morse.App.

Вопрос 4 Что такое MVC?

MVC – это архитектурный паттерн, который разделяет данные и предметную логику, графический интерфейс и обработку действий пользователя. В данной работе роль модели выполняет класс Model, роль представления – классы View и InputDialog, а роль контроллера – класс Controller.

Вопрос 5 Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление, поэтому после её изменения контроллер должен отдельно обновить интерфейс. Активная модель сама сообщает представлению об изменении своего состояния. В моём приложении Model реализует активную модель: после расчёта численности населения она напрямую вызывает view.updateResult(), и главное окно обновляет метку с результатом без участия контроллера.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности IntelliJ IDEA: редактирование, сборка, запуск и отладка Java-приложения, а также средства работы с UML и текстовой нотацией SysML v2.

По варианту №7 разработана программа «Калькулятор населения мира» с графическим интерфейсом на Swing. Программа принимает дату, проверяет её корректность, запрещает ввод дат в будущем и вычисляет население на основе исторических данных методом линейной интерполяции. Реализовано сохранение последнего ввода и его восстановление при повторном запуске.

Компоненты разделены по паттерну MVC. Реализована активная модель: после расчёта Model напрямую вызывает view.updateResult() и обновляет интерфейс без участия контроллера.

Все поставленные задачи выполнены в полном объёме.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода приложения

InputDialog.java

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionListener;

public class InputDialog extends JDialog {
    private final JTextField dateField;
    private final JButton calculateButton;

    public InputDialog(JFrame parent) {
        super(parent, "Ввод данных", true);
        setSize(350, 150);
        setLayout(new FlowLayout());
        setLocationRelativeTo(parent);

        add(new JLabel("Введите дату (ДД.ММ.ГГГГ):"));

        dateField = new JTextField(15);
        add(dateField);

        calculateButton = new JButton("Рассчитать");
        add(calculateButton);

        setVisible(false);
    }

    public void setCalculateListener(ActionListener listener) {
        calculateButton.addActionListener(listener);
    }

    public String getDateInput() {
        return dateField.getText();
    }

    public void setDateInput(String date) {
        dateField.setText(date);
    }

    public void showDialog() {
        setVisible(true);
    }

    public void closeDialog() {
        setVisible(false);
    }

    public void showError(String message) {
        JOptionPane.showMessageDialog(this, message, "Ошибка",
JOptionPane.ERROR_MESSAGE);
    }
}
```

View.java

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionListener;
```

```
public class View extends JFrame {
    private final JButton inputButton;
    private final JLabel resultLabel;
    private final JLabel lastInputLabel;

    public View() {
        setTitle("Калькулятор населения мира");
        setSize(500, 250);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());

        add(new JLabel("Нажмите для ввода даты:"));

        inputButton = new JButton("Ввести данные");
        add(inputButton);

        resultLabel = new JLabel("Население: ");
        add(resultLabel);

        lastInputLabel = new JLabel("Последний ввод: ");
        add(lastInputLabel);

        loadSavedData();
        setVisible(true);
    }

    public void setInputButtonListener(ActionListener listener) {
        inputButton.addActionListener(listener);
    }

    public void updateResult(long population) {
        String formatted = String.format("%,d", population);
        resultLabel.setText("Население: " + formatted + " человек");
    }

    public void updateLastInput(String date) {
        lastInputLabel.setText("Последний ввод: " + date);
    }

    public void saveData(String date, String result) {
        try (java.io.FileWriter fw = new
java.io.FileWriter("last_data.txt")) {
            fw.write(date + "\n" + result);
        } catch (Exception e) {
            // Игнорируем ошибки сохранения
        }
    }

    public void loadSavedData() {
        try (java.io.BufferedReader br = new java.io.BufferedReader(new
java.io.FileReader("last_data.txt"))) {
            String date = br.readLine();
            String result = br.readLine();
            if (date != null && !date.isEmpty()) {
                lastInputLabel.setText("Последний ввод: " + date);
            }
            if (result != null && !result.isEmpty()) {
                resultLabel.setText("Население: " + result + "
человек");
            }
        } catch (Exception e) {
            // Файла нет - игнорируем
        }
```

```
    }
}
```

Controller.java

```
public class Controller {
    private String lastDate = "";

    public Controller(View view, Model model, InputDialog dialog) {
        view.setInputButtonListener(e -> {
            if (!lastDate.isEmpty()) {
                dialog.setDateInput(lastDate);
            }
            dialog.showDialog();
        });

        dialog.setCalculateListener(e -> {
            try {
                String date = dialog.getDateInput();
                if (date == null || date.trim().isEmpty()) {
                    dialog.showError("Пожалуйста, введите дату!");
                    return;
                }

                lastDate = date;

                long population = model.calculatePopulation(date);

                String formatted = String.format("%,d", population);
                view.updateResult(population);
                view.updateLastInput(date);
                view.saveData(date, formatted);

                dialog.closeDialog();

            } catch (Exception ex) {
                dialog.showError(ex.getMessage());
            }
        });
    }
}
```

Model.java

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.*;

public class Model {
    private static final TreeMap<Integer, Long> HISTORICAL_DATA = new
TreeMap<>();

    static {
        HISTORICAL_DATA.put(1600, 500_000_000L);
        HISTORICAL_DATA.put(1700, 650_000_000L);
        HISTORICAL_DATA.put(1800, 950_000_000L);
        HISTORICAL_DATA.put(1900, 1_600_000_000L);
        HISTORICAL_DATA.put(1950, 2_500_000_000L);
        HISTORICAL_DATA.put(1959, 3_000_000_000L);
        HISTORICAL_DATA.put(1974, 4_000_000_000L);
        HISTORICAL_DATA.put(1987, 5_000_000_000L);
        HISTORICAL_DATA.put(1998, 6_000_000_000L);
        HISTORICAL_DATA.put(2011, 7_000_000_000L);
        HISTORICAL_DATA.put(2025, 8_200_000_000L);
```

```
    }

    public long calculatePopulation(String dateStr) throws Exception {
        LocalDate inputDate = parseDate(dateStr);
        int year = inputDate.getYear();

        LocalDate today = LocalDate.now();
        if (inputDate.isAfter(today)) {
            throw new Exception("Нельзя вводить даты в будущем! Сегодня:
" + today);
        }

        if (HISTORICAL_DATA.containsKey(year)) {
            return HISTORICAL_DATA.get(year);
        }

        Map.Entry<Integer, Long> lower =
HISTORICAL_DATA.floorEntry(year);
        Map.Entry<Integer, Long> upper =
HISTORICAL_DATA.ceilingEntry(year);

        if (lower == null) {
            return HISTORICAL_DATA.firstEntry().getValue();
        }
        if (upper == null) {
            return extrapolateForward(inputDate);
        }

        return interpolateByDay(inputDate, lower, upper);
    }

    private long interpolateByDay(LocalDate inputDate,
Map.Entry<Integer, Long> lower, Map.Entry<Integer, Long> upper) {
        LocalDate lowerDate = LocalDate.of(lower.getKey(), 1, 1);
        LocalDate upperDate = LocalDate.of(upper.getKey(), 1, 1);

        long totalDays = ChronoUnit.DAYS.between(lowerDate, upperDate);
        long daysPassed = ChronoUnit.DAYS.between(lowerDate, inputDate);

        if (daysPassed < 0) return lower.getValue();
        if (daysPassed > totalDays) return upper.getValue();

        long popDiff = upper.getValue() - lower.getValue();
        double fraction = (double) daysPassed / totalDays;

        return lower.getValue() + (long)(popDiff * fraction);
    }

    private long extrapolateForward(LocalDate date) {
        LocalDate baseDate = LocalDate.of(2025, 1, 1);
        long basePop = HISTORICAL_DATA.get(2025);
        long daysAfter = ChronoUnit.DAYS.between(baseDate, date);
        if (daysAfter < 0) daysAfter = 0;
        long growthPerDay = 219_178L;
        return basePop + (daysAfter * growthPerDay);
    }

    private LocalDate parseDate(String dateStr) throws Exception {
        String[] parts = dateStr.split("\\.");
        if (parts.length != 3) {
            throw new Exception("Дата должна быть в формате
ДД.ММ.ГГГГ");
        }
```

```
        int day = Integer.parseInt(parts[0]);
        int month = Integer.parseInt(parts[1]);
        int year = Integer.parseInt(parts[2]);

        if (day < 1 || day > 31 || month < 1 || month > 12) {
            throw new Exception("Некорректная дата");
        }

        try {
            return LocalDate.of(year, month, day);
        } catch (Exception e) {
            throw new Exception("Некорректная дата (например, 31 апреля
не существует)");
        }
    }
}
```

Main.java

```
public class Main {
      public static void main(String[] args) {
        View view = new View();
        Model model = new Model();
        InputDialog dialog = new InputDialog(view);
        new Controller(view, model, dialog);
      }
}
```